Exercise 1

Submitted by

Name: Kamrul Hasan Ovi

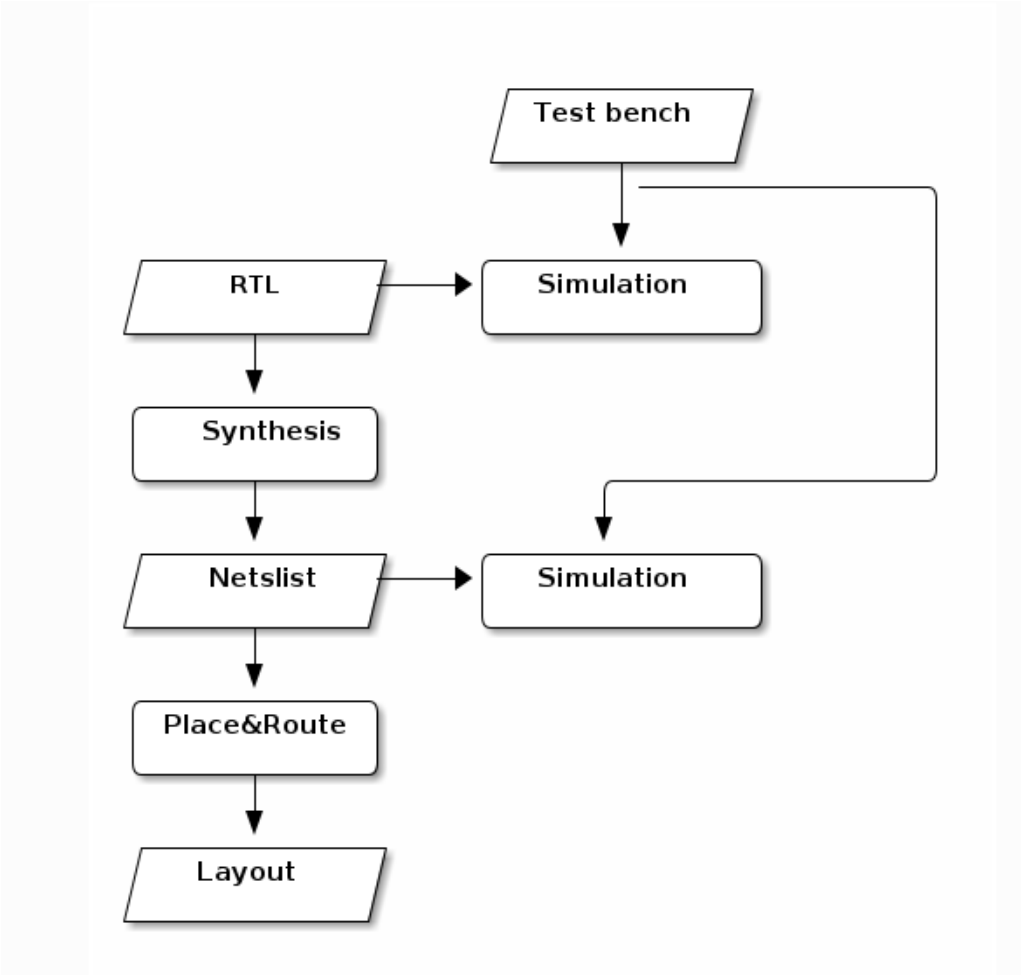
Matriculation: 6335212

1 Goal

The goal of this lab is to gain a complete overview of the IC design process. This lab includes:

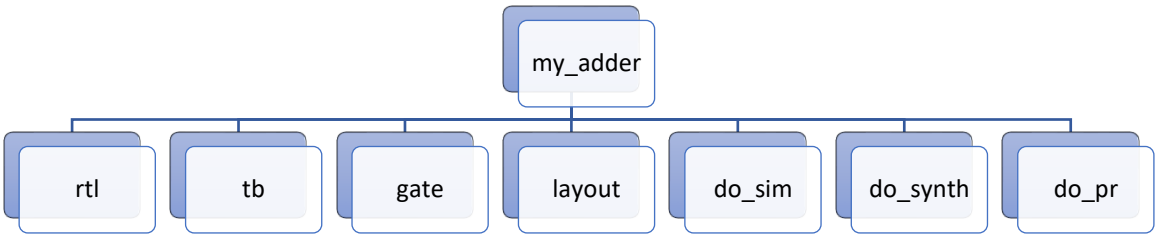
- RTL design and simulation
- Synthesis
- Simulation of the gate-level netlist with back-annotated delays
- Place and Route

The following figure shows the general overview of the lab.



1.1 Organizing data

For designing an IC, we have to perform several steps to complete the design and observe the data obtained from each step. To make it easier, we organized the generated data by creating folders and subfolders. Here is the overview of the folders.




```
[l_ids309@it002057 ~/work]$ ll my_adder
insgesamt 28
drwxr-xr-x 2 l_ids309 agids1lab 4096 28. Apr 10:51 do_pr
drwxr-xr-x 5 l_ids309 agids1lab 4096 28. Apr 11:43 do_sim
drwxr-xr-x 7 l_ids309 agids1lab 4096 28. Apr 12:12 do_synth
drwxr-xr-x 2 l_ids309 agids1lab 4096 28. Apr 10:50 gate
drwxr-xr-x 2 l_ids309 agids1lab 4096 28. Apr 10:50 layout
drwxr-xr-x 2 l_ids309 agids1lab 4096 28. Apr 10:55 rtl
drwxr-xr-x 2 l_ids309 agids1lab 4096 28. Apr 2026  tb
```

2 Design the RTL and Test bench

To start the design, first we had to write the `RTL` code and to test it, we wrote the test bench under the `my_adder` folder and into the `rtl` and `tb` sub folder.

The rtl code and the test bench code shown below

```
library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;

  entity my_adder is

    port (
      clk,rstn: in std_logic;
      dta,dtb : in  std_logic_vector(7 downto 0);
      dto : out std_logic_vector(7 downto 0));

  end my_adder;

  architecture rtl of my_adder is

    signal dta_int,dtb_int,dto_int : std_logic_vector(7 downto 0);

  begin  -- architecture rtl

    -- Define the memory elements
    process (clk, rstn) is
    begin  -- process
      if rstn = '0' then                -- asynchronous reset (active low)
        dta_int <= (others=>'0');
        dtb_int <= (others=>'0');
        dto     <= (others=>'0');
      elsif clk'event and clk = '1' then  -- rising clock edge
        dta_int <= dta;
        dtb_int <= dtb;
        dto     <= dto_int;
      end if;
    end process;

    -- Define the adder
    dto_int <= std_logic_vector( signed(dta_int)+signed(dtb_int) );

  end architecture rtl;
```

```
library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;

  entity my_adder_tb is
  end entity my_adder_tb;

  architecture tst of my_adder_tb is

    -- component ports
    signal dta, dtb : std_logic_vector(7 downto 0);
    signal dto      : std_logic_vector(7 downto 0);

    -- clock
    signal clk : std_logic := '1';
    signal rstn : std_logic;

  begin  -- architecture tst

    -- component instantiation
    DUT: entity work.my_adder
      port map (
        clk => clk,
        rstn => rstn,
        dta => dta,
        dtb => dtb,
        dto => dto);

    -- clock generation
    clk <= not clk after 1 ns;
    rstn <= '0', '1' after 500 ps;

    -- waveform generation
    WaveGen_Proc: process
    begin
      dta <= std_logic_vector(to_signed(3,  dta'length));

      dtb <= std_logic_vector(to_signed(-1, dtb'length));
      wait until clk = '0';
      wait until clk = '0';
      dta <= std_logic_vector(to_signed(6,  dta'length));
      dtb <= std_logic_vector(to_signed(1, dtb'length));
      wait until clk = '0';
      dta <= std_logic_vector(to_signed(-4,  dta'length));
      dtb <= std_logic_vector(to_signed(-3, dtb'length));
      wait until clk = '0';
```

```
    dta <= std_logic_vector(to_signed(2**7-1,  dta'length));
    dtb <= std_logic_vector(to_signed(1,  dtb'length));
    wait until clk = '0';
    wait until clk = '0';

    report "End simulation" severity failure;  -- Finish
  end process WaveGen_Proc;

end architecture tst;
```

3 Compile and simulate the codes

For compile and simulate, we used the `irun` tool from Cadence. By compile, we make sure the codes don't have any error. For compilation we used the command

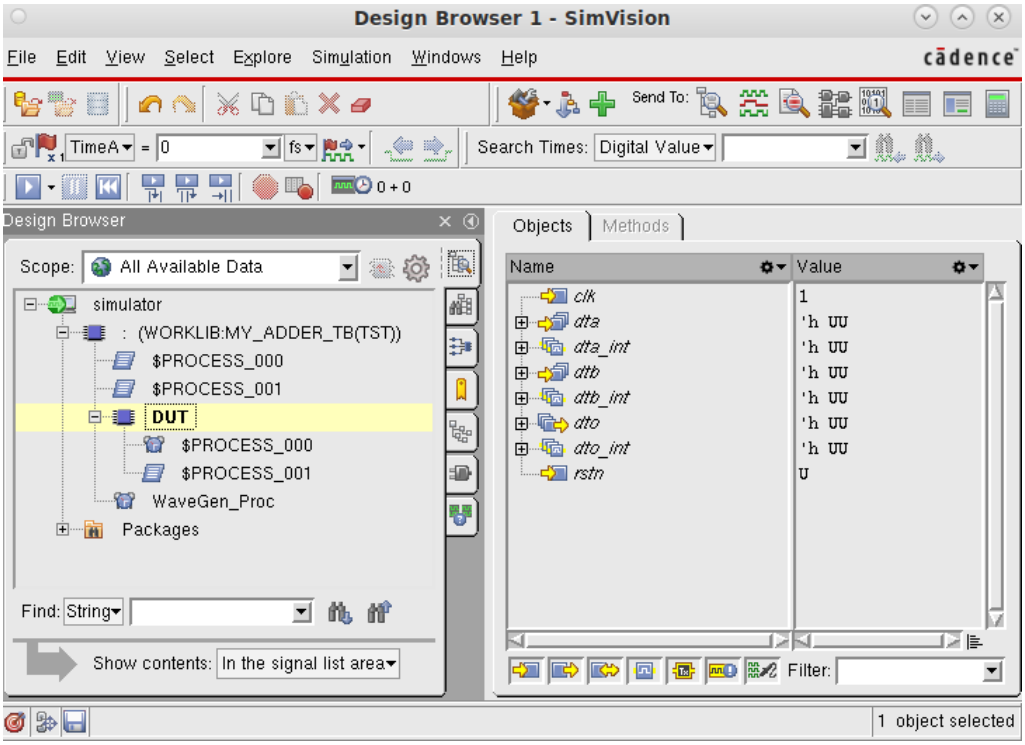
```
irun -compile ../rtl/my_adder.vhd

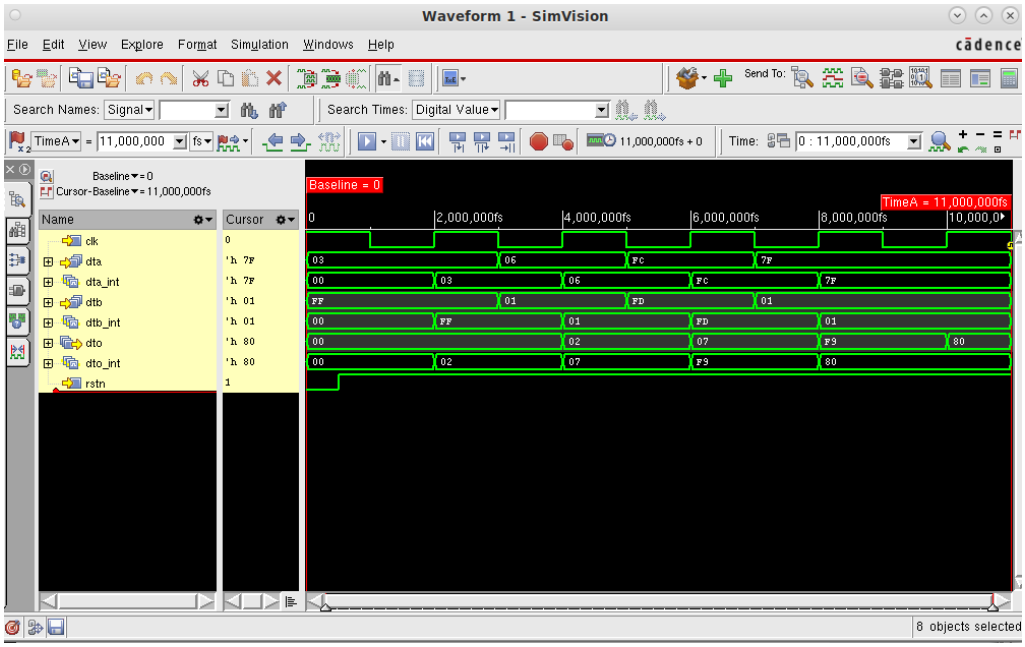
irun -compile ../tb/my_adder_tb.vhd
```

```
[l_ids309@it002057 do_sim]$ irun -compile ../rtl/my_adder.vhd
TOOL:   xrun(64)        24.03-s004: Started on Apr 28, 2026 at 12:18:16 CEST
xrun(64): 24.03-s004: (c) Copyright 1995-2024 Cadence Design Systems, Inc.
xrun: *W,HVDEPRREN: HDL.VAR Variable (IRUNOPTS) is deprecated. Use (XRUNOPTS) in
stead.
xrun: *W,NCEXDEP: Executable (irun) is deprecated. Use (xrun) instead.
TOOL:   xrun(64)        24.03-s004: Exiting on Apr 28, 2026 at 12:18:17 CEST  (t
otal: 00:00:01)
```

By simulate, we checked our designed code work perfectly or not. We checked the waveform for different inputs values. For simulation, we used the command

```
irun  -gui  ../tb/my_adder_tb.vhd  ../rtl/my_adder.vhd -top my_adder_tb
```





4 Synthesis

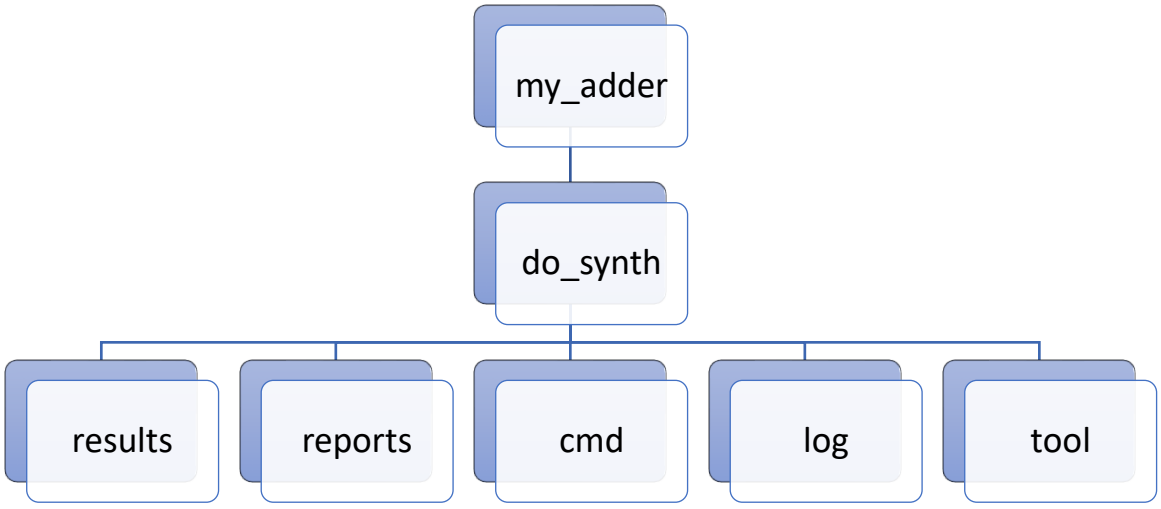
The goal of synthesis is to convert the RTL code into a gate-level netlist. The netlist describes the implementation of the unit using digital gates from an existing library of standard cells. For synthesis, we used the `design Vision` tool from Synopsis.

Input required for synthesis:
• RTL code
• SDC file
• Library file
• LEF file

Output from synthesis:
• Netlist
• SDC file
• Report

4.1 Setup for the synthesis

During the synthesis process, we generate a lot of data. Therefore, it is a good idea to organize the information into sub-directories.



Before starting the synthesis tool, we created a .synopsys_dc.setup file to select the standard cell library and defined it in log file directories. This helps to organize all generated data properly.

```
define_design_lib work -path ./tool/work

set_app_var  view_log_file        ./log/synth_view.log

set_app_var  sh_command_log_file  ./log/synth_sh.log

set_app_var  filename_log_file    ./log/synth_file.log
```

```
set_app_var search_path       [concat ./cmd/  [get_app_var search_path] ]
```

then, we defined the library path and `chose NangateOpenCellLibrary_typical_ccs.db` as the target library, which contains typical delay values.

```
set library_path "/usr/prog/optech/freepdk/fpdk45_v20__IDS/LIB/"

set library_name "NangateOpenCellLibrary_typical_ccs.db"

set_app_var target_library    $library_name

set_app_var link_library      [concat $library_name dw_foundation.sldb "*"]

set_app_var search_path       [concat $library_path [get_app_var search_path] ]

set_app_var synthetic_library [list dw_foundation.sldb]

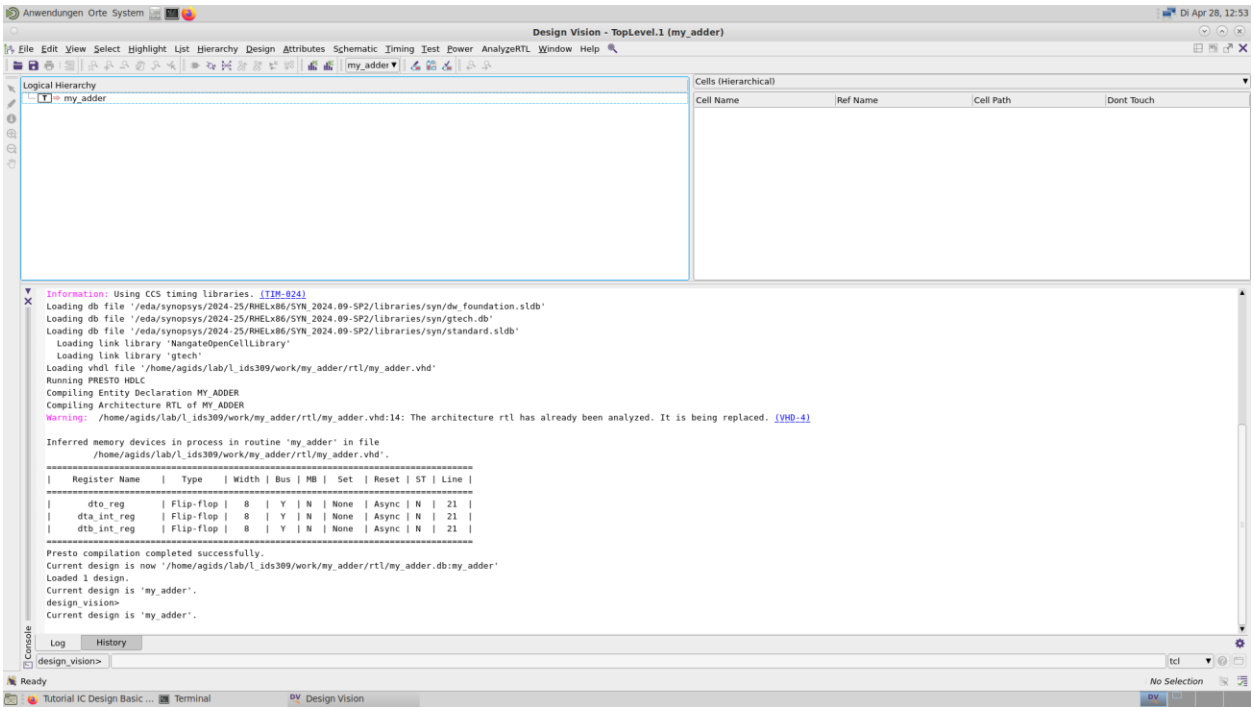
set_app_var symbol_library    [list class.sdb]

set_app_var  vhdlout_use_packages {ieee.std_logic_1164.all
NangateOpenCellLibrary.Components.all }

set_app_var vhdlout_write_components FALSE
```

4.2 Synthesis Process

- First we ran the command `design_vision | tee log/synthesis.log &` for launch the tool. Here `design_vision` command executes to launch the tool and tee log helps to save the log file to the log directory and it would save by name of synthesis.log
- In the Design Vision prompt, we ran the command `read_file -format vhdl {../rtl/my_adder.vhd}` to read the RTL file that we wanted to synthesize. After executing this command successfully, the tool showed some reports.



- Then, we defined the design constraints using TCL commands such as clock definition, input/output delay, clock uncertainty, and timing checks.

```
create_clock [get_ports clk]  -period 8.0  -waveform {0 4} -name clk

set_clock_uncertainty 0.025  -setup [get_clocks clk]

set_clock_uncertainty 0.025  -hold [get_clocks clk]

set_clock_transition -fall 0.04 [get_clocks clk]

set_clock_transition -rise 0.04 [get_clocks clk]

set_dont_touch clk

set_dont_touch rstn

set_clock_latency -max -source 0.1 [get_clocks clk]

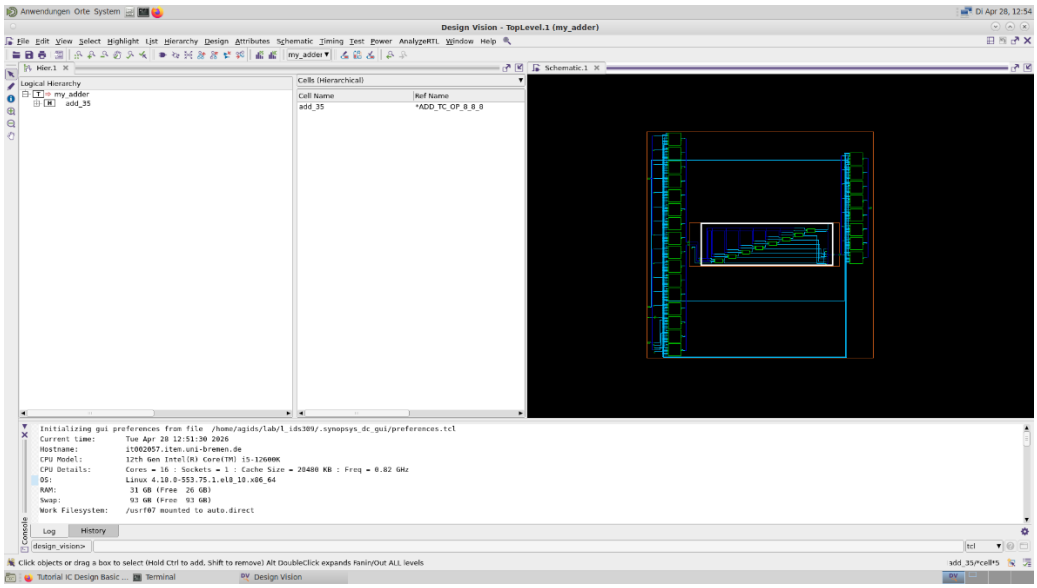
set_input_delay -max -clock clk  0.05 [get_ports {dta dtb}]

set_output_delay -max -clock clk  0.05 [all_outputs]

set_false_path -from [get_ports rstn]

check_timing
```

- After that, we mapped the design into gates using the compile command. The synthesis results were generated after compilation.



- Next, we generated reports using commands like `report_area` and `report_power`, and saved them in the report's directory.

```
****************************************
Report : area
Design : my_adder
Version: W-2024.09-SP2
Date   : Tue May  5 10:15:49 2026
****************************************

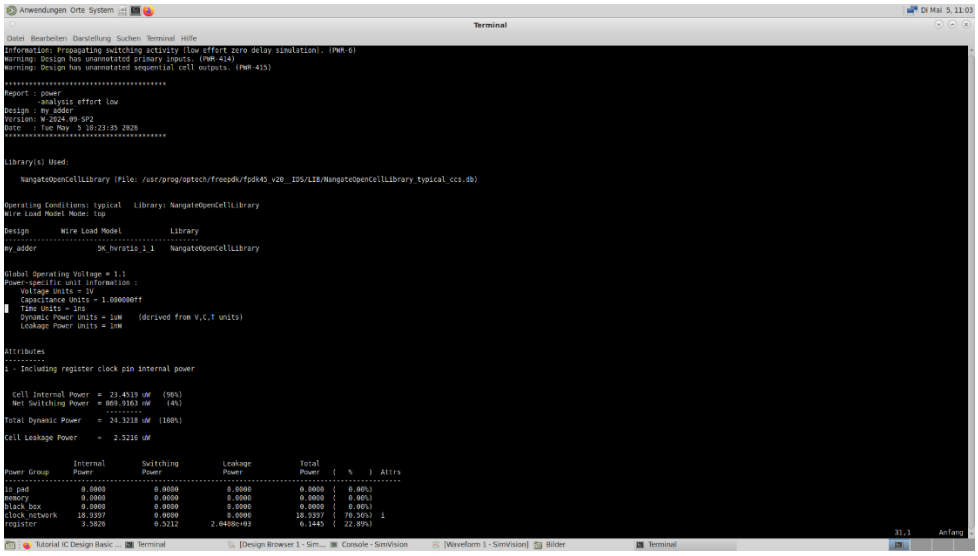
Information: Updating design information... (UID-85)
Library(s) Used:

    NangateOpenCellLibrary (File: /usr/prog/optech/freepdk/fpdk45_v20__IDS/LIB/N
angateOpenCellLibrary_typical_ccs.db)

Number of ports:                           52
Number of nets:                            82
Number of cells:                           35
Number of combinational cells:              9
Number of sequential cells:                24
Number of macros/black boxes:               0
Number of buf/inv:                          0
Number of references:                       2

Combinational area:                32.452000
Buf/Inv area:                       0.000000
Noncombinational area:            127.680004
Macro/Black Box area:               0.000000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                  160.132004
Total area:                 undefined
1
~
```



- Finally, we saved the netlist using write_file command in both VHDL and Verilog formats after applying change_names rules.

  change_names -hier -rules vhdl
  change_names -hier -rules verilog
  write_file -hierarchy -f vhdl -output "./results/my_adder.vhd"
  write_file -hierarchy -f verilog -output "./results/my_adder.v"

5 Simulate gate-level

5.1 Setup gate-level simulation

For gate-level simulation, we need the standard cell library. We created a cds.lib file and write:

DEFINE nangateopencelllibrary
/usr/prog/optech/freepdk/fpdk45_v20__IDS/VITAL/nangateopencelllibrary

5.2 Simulate with a SDF file

We compiled the sdf file before simulating the delays to observe is there any error in the sdf file.

```
ncsdfc ../gate/my_adder.sdf  -o  my_adder.sdf.X
```

Then, we created a file sdf.cmd indicating the delay type. For this design we used the worst-case delays.

```
COMPILED_SDF_FILE = "my_adder.sdf.X",

SCOPE = :DUT,

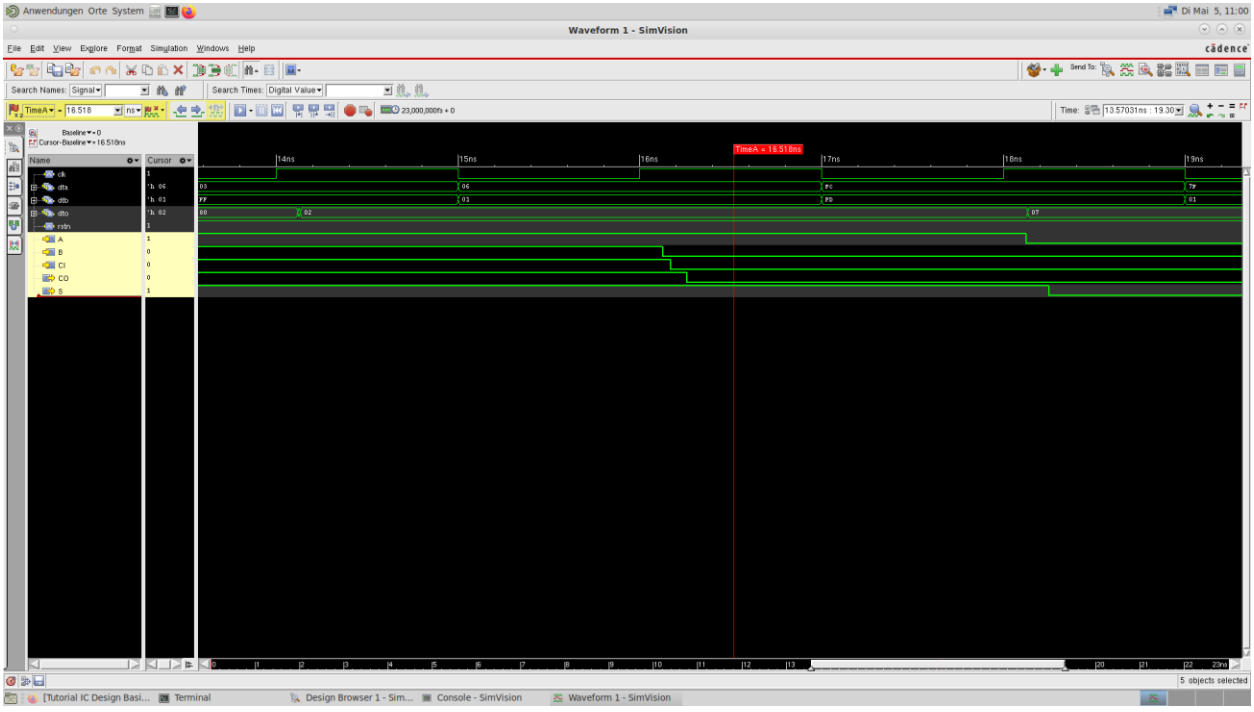
MTM_CONTROL = "MAXIMUM",

SCALE_FACTORS = "1.0:1.0:1.0";
```

Thereafter, we run the test-bench with issuing a netlist with back-annotated delays

```
irun -compile ../gate/my_adder.vhd

irun -gui ../tb/my_adder_tb.vhd  -top my_adder_tb  -sdf_cmd_file sdf.cmd
```



5.3 Power estimation

To get the power estimation we dumped a file format called saif and execute it

```
dumpsaif -depth to_cells -output my_adder.saif -scope :DUT

run

irun ../gate/my_adder.vhd  ../tb/my_adder_tb.vhd  -top my_adder_tb:tst  -sdf_cmd_file sdf.cmd -input do_sim_saif.tcl
```

After getting the saif file, we got back to the back-annotated power report. To do this , we go back to the do_synth directory and launch the `Design_vision`, and run the following commands

```
cd ../do_synth

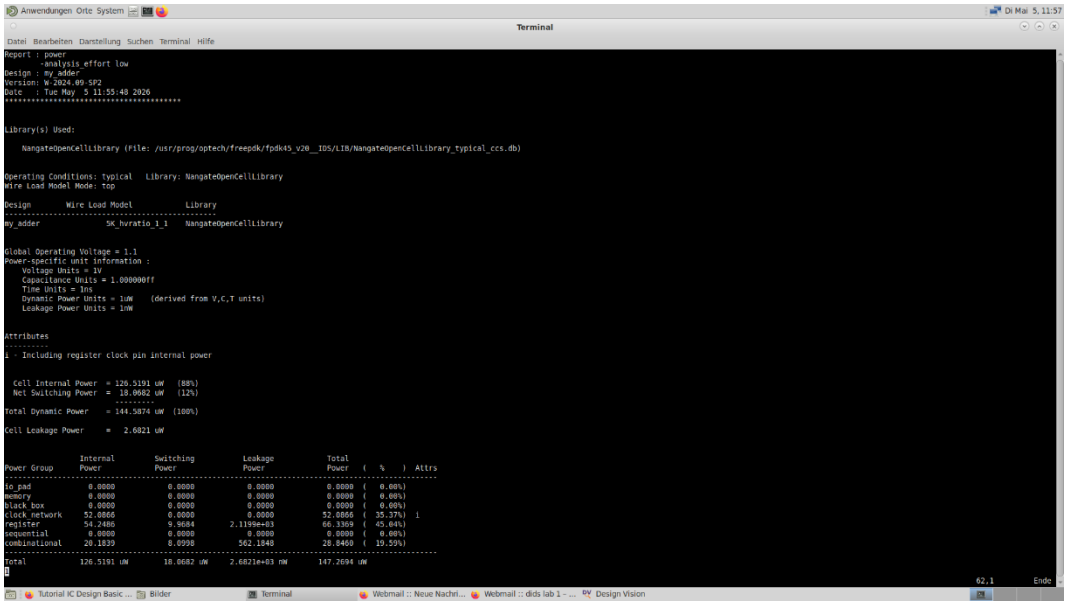
design_vision | tee log/synthesis.log &

read_file -format ddc {./results/my_adder.ddc}

read_saif -input ../do_sim/my_adder.saif -instance_name DUT -verbose

report_power > reports/my_adder-post_synth.power
```

Here is the new report



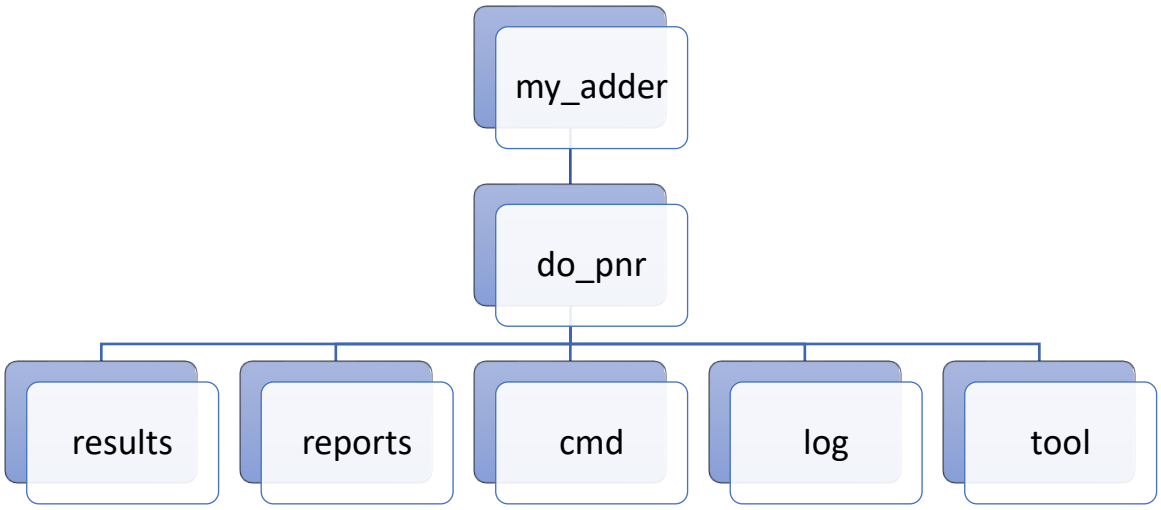
6 Place and Route

The goal of the place and route is to transform the netlist into a layout . The steps including

- Design initialization
- Floorplaning
- Placement
- Routing
- Export design

6.1 Setup the tool

To organize the data from different stages, we created subdirectories inside the `do_pnr` folder.



Then we started the tool using the command

```
innovus -log log/
```

6.2 Design Initialization

In this stage, we read the design files, LEF file, and define power and ground nets:

```
set init_lef_file ../../0_FreePDK45/LEF/NangateOpenCellLibrary.lef

set init_gnd_net VSS

set init_pwr_net VDD

set init_verilog ../gate/my_adder.v

set init_top_cell my_adder

init_design
```

6.3 Floorplan

In this step, we defined the core ratio and the distance between the core and I/O boundary:

```
floorPlan  -r 1 0.7 3.0 3.0 3.0 3.0
```

6.4 Place the cells and pins

Here, we placed all the standard cells and I/O pins:

```
setPlaceMode -placeIOPins 1

placeDesign -noPrePlaceOpt
```

After placement, we filled the blank space adding filler cell into the design,

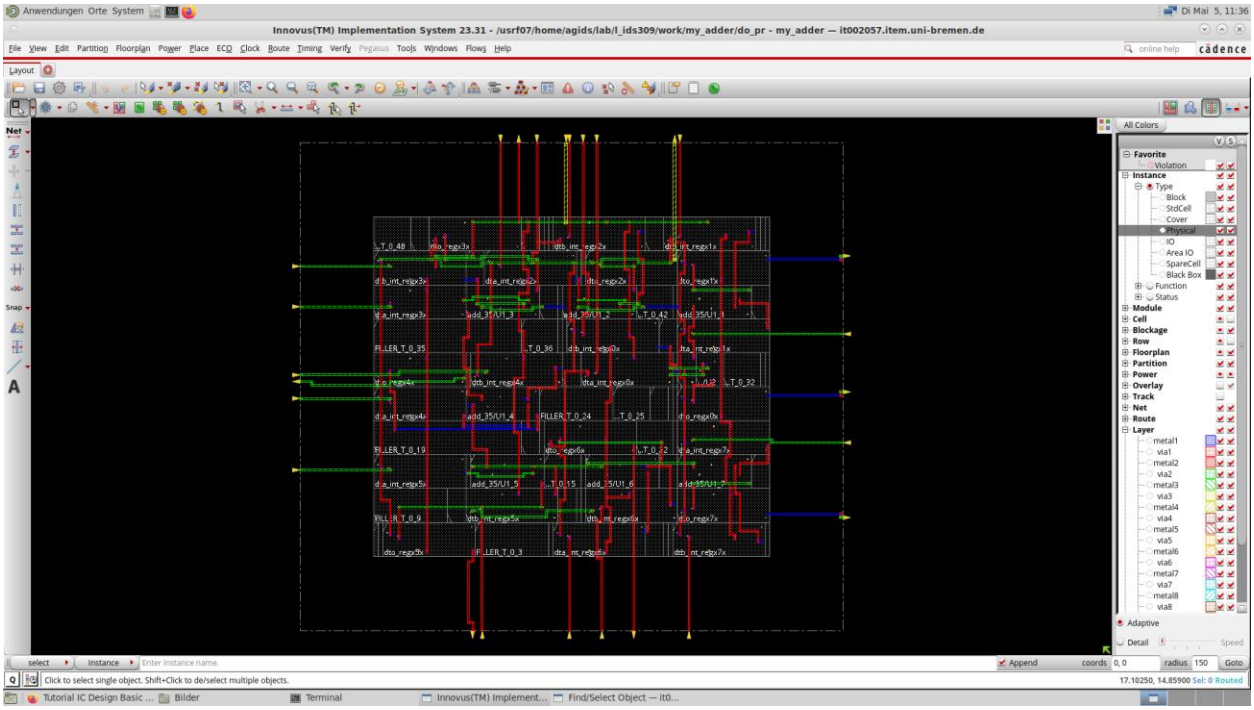
```
addFiller -cell FILLCELL_X8 FILLCELL_X4 FILLCELL_X2 FILLCELL_X1 -prefix FILLER
```

6.5 Route the design

After placement, routing is done to connect all the cells:

```
routeDesign -globalDetail
```

This step creates all the physical connections using metal layers. The final outlook of the design showing below



6.6 Verify and write result

After successfully routing the design we checked the drc error and also checked the power report by using the commands

```
verify_drc -report reports/my_adder.drc

verify_connectivity -report reports/my_adder.connect
```

Finally, we exported the layout into gds file

```
streamOut results/my_adder.gds -mapFile streamOut.map -libName my_library -units 2000 -mode ALL
```